# JPA Day 2 — 2 Main Tasks

Based on the uploaded Day 2 material: Relationships, Fetching, Cascading, Inheritance, JPQL/HQL, Criteria API & JPA

## Task 1 — Build a Library Management System

Build a small Library Management System using JPA and H2. The goal is to practice entity relationships, ownership, fetching, cascading, and inheritance in one project.

### Requirements

- Create **Author** and **Book** entities with a bidirectional @OneToMany / @ManyToOne relationship.
- Make Book the owning side and store the FK as author_id.
- Use mappedBy = "author" on Author.books.
- Add cascade = CascadeType.ALL and orphanRemoval = true.
- Create an addBook() helper that keeps both sides synchronized.
- Create a **Publisher** entity and connect Book → Publisher using @ManyToOne.
- Create a **Category** entity and connect Book ↔ Category using @ManyToMany with a join table.
- Create an **Employee** / **Customer** inheritance hierarchy using one JPA inheritance strategy of your choice. Then explain why you chose it.
- Configure persistence.xml with an H2 in-memory database and all required entities.
- Use LAZY fetching for collections and explicitly configure the to-one relationship where appropriate.
- Insert realistic sample data and inspect the generated database tables and foreign keys.

### What you should be able to demonstrate

- Identify the owning and inverse sides of every relationship.
- Explain where each foreign key is stored and when a join table is required.
- Demonstrate what happens when a Book is removed from Author.books with orphanRemoval = true.
- Demonstrate cascading when persisting an Author with new Books.
- Show one intentional LazyInitializationException and explain why it happens.

## Task 2 — Query and Analyze the Library

Using the same project, practice JPQL/HQL, JOIN FETCH, aggregation, and Criteria API.

### Requirements

- Find all books by a given author's name using JPQL.
- Find all books belonging to a given publisher.
- Find a specific Book by id using a positional parameter.
- Fetch an Author together with all of their Books using JOIN FETCH.
- Write an aggregate query that returns each author's name and the number of books they have using COUNT and GROUP BY.
- Create a Criteria API query that finds books by title.

- Extend the Criteria API query so that the title and author-name filters are optional and predicates are added dynamically.
- Compare a normal LAZY query with the JOIN FETCH version and explain why JOIN FETCH can prevent LazyInitializationException for that use case.
- Write one query using standard JPQL and explain how a Hibernate-specific HQL feature would differ.
- As an optional extension, move the Author–Book relationship mapping to orm.xml and verify that the runtime mapping remains equivalent.

## Final deliverable

- A working JPA project with the complete domain model.
- Sample data inserted into H2.
- All required JPQL and Criteria queries.
- A short README explaining the relationship mappings, owning sides, fetch choices, cascade choices, inheritance strategy, and query decisions.